GENERATIVE AI & AGENTS

AI · ML · DS / REVISION NOTES

COURSE NOTES

Build Your AI Agent with LangGraph

Building an agent from first principles

9
SECTIONS

9
PAGES

Compiled October 2026
github.com/ankitgupta1729/AI-ML-DS

Contents

2 Two types of language modeling tasks 3
3 Modern LLMs do 3
6 LLM training process 4
8 Tokenization 5
9 Embedding Math 6
LECTURE 10 Context and Tokens 6
11 Problem statement for LLMs 7
12 Context Length 7
15 RAG Pipeline 8
16 Popular vector databases 8

1. A Model, in general, is something that simulates or represents something. For example, a weather model is something that simulates the weather. It predicts the weather means it keeps track of how the weather has been so far means it checks the pattern in past and tells what it likely to be in the future.

A language model does the similar thing. A language model does modeling the language. A language model captures and simulates what the language is.

Chat is a text generation process where we generate dialogues and using LLM, we do the text generation which is basically the predicting the text. They predict the likelihood of the next set of tokens given the previous set of tokens.

TOPIC 2

Two types of language modeling tasks

A. Autoencoding tasks
B. Autoregressive tasks

- An autoencoder is a type of neural network architecture designed to efficiently compress(encode) the input data down to its essential features and then reconstruct(decode) the original input from this compressed representation.
- A statistical model is autoregressive if it predicts future values based on past values. For example, an autoregressive model might seek to predict a stock's future prices based on its past performance.

Autoregressive models used to predict next words in a sequence.

TOPIC 3

Modern LLMs do

- Text prediction machines
- Recursive completion means if predicts next word then based on this it predicts next-to-next word and so on like we can predict the weather for entire year if we can predict the weather for next day
- Generate text based on parameters like temperature, top_k, top_p, frequency_penalty, presence_penalty etc.
- Knows what to say next based on training data

4. Sometimes LLM gives results based on latest data. It is not doing a web crawling but instead using RAG with embeddings or use function or tool calling.
5. LLM can give wrong answer because it works not probabilities and not facts and when you provide the wrong data it can give wrong results. Also it hallucinates when next high probable word is also wrong.

TOPIC 6

LLM training process

It contains the following steps:

A. Pre-training
B. Instruct tuning
C. Finetuning

- Pretraining is the initial phase of training an LLM where it learns from a large, diverse dataset of often trillions of tokens. The goal of this step is to develop a broad understanding of language, context and various types of knowledge.

The more text you show to a model a more better understanding it gets about the language.

Every LLM is pre-trained and training happens on large corpus of text. The LLM "learns" and "understands" the text. It adjusts its internal weights based on the learnings.

- Instruct Tuning:

Suppose we are using GPT-3 model which does the autocompletion and when I give the prompt as `Explain the moon landing to a 6 years old child in a few sentences` then GPT-3 gives the answer as `Explain the theory of gravity to a 6 years old. Explain the theory of relativity to a 6 years old in few sentence. Explain the big bang theory to a 6 years old. Explain the evolution to a 6 years old.`

So, GPT-3 is not answering the question instead doing auto-completion. This is the problem with autocompletion, it doesn't response. So, we need more than text completion. So, we need to train the model on conversation which means we have to do instruct tuning which is sometimes called `transfer learning`.

Here, we have to train a model to follow instructions using an instruct dataset: prompt-response pairs.

Here Dataset is manually curated and trained.

We can also train LLM using reinforcement learning with human feedback(RLHF) which is a powerful way to train the model. Here we use like(reward)/unlike(penalty) and we do over and over and model learns it.

- Finetuning:

Here we further train a pre-trained model but we train for a more specific dataset. This finetuning step is optional. ChatGPT probably comes with only pre-training and instruct tuning only and not with finetuning. Here we typically use a focused and smaller dataset and businesses do it for its own use cases.

Here we can make our own chatgpt by giving our own data in input-output pairs and train it. We have done it in some other repo previously.

7. What are tokens ?

Tokens are mathematical representation of text. We store text as digital numbers/ASCII/unicode etc. Each character gets a number.

TOPIC 8

Tokenization

- Process of breaking down text into tokens.
- Each token is assigned a number.
- Model is given a series of tokens.
- It outputs a series of tokens.

The Process:

Text --> Token Encoding --> LLM --> Token Decoding --> Text

Just go to a token visualizer (https://huggingface.co/spaces/Xenova/the-tokenizer-playground) and put text as `Hi, how are you doing?` or `Tokenization` and see how embedding models convert text to numerical values.

- How is text broken down like Tokenization = Token + ization?

-- It is not grammatical
-- It is based on statistical significance and optimization of frequencies and usage. For example, comma is a token because in different context it used differently.
-- Multiple approaches from multiple tokenizers.

One token per word ?

-- Per character is meaningless
-- Per word requires too many tokens
-- Doesn't convey meanings and relationships (e.g. "teach" vs "teachings")

Remember that same tokenizer is used for both training and inference.

So, text is broken down based on frequencies of occurrence. Something has more frequency has its own token and something occurred less frequently is broken down and see if any portion of it already has a token and then it matches that or if portion occurs in more less occuring words then it is assigned to that token. (Byte pair encoding)

- what number it assigned to and who assigned it ?

This is based on embeddings or vectors. To identify something as a cat, "whisker" feature can be used and some number assigned to it.

Every token is a embedding vector in which each value represents the learned weights for each feature.

TOPIC 9

Embedding Math

v("king") - v("man") + v("woman") =~ v("queen")

here, v represents the embedding vector.

It makes sense because in n-dimensional space, distance between the man and king is same as distance between the man and queen.

Similarly,

v("Paris") - v("France") + v("Italy") =~ v("Rome")
v("walking") - v("walk") + v("run") =~ v("running")
v("good") - v("bad") + v("happy") =~ v("joyful") or similar
v("summer") - v("hot") + v("cold") =~ v("winter")

LLMs derive meanings using embeddings.

Lecture 10 · Context and Tokens

"Rare words always stay as one token" -- False

Because rare words get broken into smaller subwords or characters so the model can still represent them.

Example, "Hello" is a common words so it is considered as one token and "tokenization" is not a common word so it gets broken into "token" and ""ization" as two tokens.

LLMs figure out how many dimensions are needed for each token. Also, more the size of the model (number of parameters), more the number of dimensions.

Embeddings capture meaning.

TOPIC 11

Problem statement for LLMs

Given a sequence of tokens $x_1,x_2,...,x_t$, predict the most likely next token x_{t+1}.

Transformer solves this problem using embeddings, word positions and ENTIRE text to generate the next word/token.

Transformer compare words in parallel and predict the next word.

Example:

A. The animal didn't cross the street because it was too tired.
B. The animal didn't cross the street because it was too wide.

In 'A', "it" refers to the animal and in 'B', "it" refers to the street.

So, here embeddings for "it" gets changed dynamically based on the context. This is called context dependent embeddings or dynamic embeddings and we don't use static embeddings. Here, we compare each word with other words in parallel and see that in first "it" will be closed to "animal" and in second "it" will be closed to "street".

TOPIC 12

Context Length

Maximum number of tokens that an LLM can process in a single interaction means what an LLM can pay attention to i.e. input+output.

LLMs have context limit.

Context Length measured in number of tokens.

Some examples:

GPT-3: 2048 tokens
GPT-4o: ~60K to 128K tokens
Claude: ~100K tokens
Llama 3.1: 128K tokens
Gemini 1.5 Pro: Upto 1M tokens

13. LLMs are stateless. LLMs don't remember anything means it doesn't have any memory but ChatGPT platform does it because here full context passed each time.

The more tokens LLM have, the less efficient it is.

14. For dynamically changing information, fine-tuning can be used but best option is RAG. RAG is also best way to handle limited context sizes.

RAG dynamically retrieved "documents".

Q) If the return policy from the company and if return policies found from the LLM training, how does it give preference to company's return policy ?

A) LLM usually gives priority to the context but sometimes training data also get prioritize over context like for security protocols, LLM follow training data instead of your instructions.

RAG's goal: Provide relevant text within context limit and add to LLM's "word's knowledge" with updated/changing info and also add specialized info.

Here, in RAG, we need embedding based search.

Vector database is a purpose-built store for embedding and their original document. It helps look up document given an ID (but not the key usage). It gets document nearest to an embedding.

Vector database stores (embedding_vector, document_or_metadata).

Here, we don't compare the query's embedding with the vector database's embedding. Instead, we use `Indexing`. Indexing is the engine builds an `ANN( Approximate Nearest Neighbor)` index. This ANN is an algorithm. It groups vectors so "nearby" points link together -- enabling $O(\log N)$ or better lookup. ANN index quickly returns the top-k most similar vectors (approximate but very fast). After that it retrieve their keys/payloads and returns the original text snippets.

TOPIC 15

RAG Pipeline

A. User Prompt --> LLM

LLM drafts a retrieval query (prompt itself or a reformulation)

B. Vector DB

Embed that query --> retrieve top-k relevant chunks

C. Context Assembly

Concatenate with the original prompt.

D. Final generation

feed full context back into the LLM.

TOPIC 16

Popular vector databases

- Pinecone
- Qdrant
- Weaviate
- Milvus

17. In Claude, "Projects" feature use the RAG service.